

Claude Code로 진짜 프로그램 만들기

묘사하면, 내 컴퓨터에서 도는 도구가 된다 · 강사 진행 대본 (정금구)

이 문서 사용법

한 페이지 = 한 슬라이드. 강의 중 한 페이지씩 넘기며 보세요.

상단 번호 박스 [01] = 슬라이드 번호.

★ KEY POINT (보라) — 이 슬라이드에서 반드시 전달할 한 문장.

이렇게 말하세요 (검은 박스) — 실제 강의 멘트. 그대로 읽어도 됨.

강사 팁 (노랑) — 현장 진행 노하우. 참고 (회색) — 배경 설명.

→ 넘어가며: 다음 슬라이드로 가는 연결 멘트.

자주 나오는 코딩 용어 (강사용 한 줄 설명)

터미널 / **Git Bash** 명령을 글로 입력하는 검은 창 · **Node.js** 프로그램을 내 컴퓨터에서 실행하는 엔진 · **로컬 서버** 내 컴퓨터 안에서만 도는 서버 · **포트** 프로그램의 '문 번호'(예: 3000) · **localhost** 내 컴퓨터 주소 · **도구(Tools)** 화면의 Read·Write·Bash 표시 · **스펙** '무엇을 받아 무엇을 내보낸다'를 적은 약속 메모 · **git** · **커밋** 작업을 시점별로 저장(되돌리기 가능) · **GitHub** 코드를 인터넷에 올려 나누는 곳

구분	시간	슬라이드	비고
오프닝 + 복습	8분	1~3	4차·R-PCCO 복습
Part 1 — 왜 Claude Code 인가	12분	4~6	
Part 2 — 만능 5단계 + 첫 버전	15분	7~9	
Part 3 — 로컬 서버 테스트	12분	10~11	
Part 4 — 미리보기 + 계획 모드	15분	12~13	
Part 5 — 디버깅	12분	14	
Part 6 — 레이어 / I-O 계약	22분	15~20	핵심
Part 7 — 재사용 스케폴드	14분	21~23	
Part 8~9 — git · 공유	20분	24~25	
실습 + 마무리	18분	26~32	

표지 — “Claude Code로 진짜 프로그램을 만든다”

1분

★ KEY POINT

오늘 90분 뒤, 묘사만으로 내 컴퓨터에서 실제로 도는 도구 하나를 만들어 돌아갑니다.

이렇게 말하세요

안녕하세요, ‘Claude Code로 진짜 프로그램 만들기’를 맡은 정금구입니다. 4차에서 claude.ai로 운세 앱 만들어보셨죠? 그건 브라우저 안에서만 돌아갔어요. 오늘은 한 단계 올라갑니다 — 내 컴퓨터에서 진짜로 실행되고, 내 파일을 다루는 프로그램요.

강사 팁

표지는 3초. 1~5차와 같은 디자인 톤으로 ‘시리즈’임을 각인. 설치는 완료 전제이니 도입부엔 점검 3분만.

→ 넘어가며: “지난 강의 잠깐 복습하고, 오늘 뭐가 달라지는지부터 봅시다.”

4차에서 만든 앱, 지금 어디 있죠?

3분

★ KEY POINT

오늘도 R-PCCO는 그대로 쓴다. 단, 맥락(C)에 ‘내 컴퓨터’가 들어갈 뿐이다.

이렇게 말하세요

질문 하나. 4차에서 만든 운세 앱, 지금 여러분 컴퓨터 어디에 있죠? (없죠.) 브라우저 닫으니 사라졌어요. 오늘 배우는 R-PCCO는 똑같습니다. 역할·목적·맥락·제약·출력. 다른 건 딱 하나, ‘맥락’에 내 컴퓨터·내 폴더·실행 환경이 들어간다는 거예요.

강사 팁

R-PCCO 다섯 칸은 청중에게 빠르게 되물어 복습. 길게 끌지 말 것 — 오늘 분량이 많습니다.

→ 넘어가며: “그래서 오늘, 세 가지가 달라집니다.”

90분 뒤, 이렇게 달라집니다

3분

★ KEY POINT

기능 체득 + 레이어 설계 + 재사용 스캐폴드. 도구 하나로 셋을 다 가져간다.

이렇게 말하세요

90분 뒤 세 가지가 달라집니다. 하나, Claude Code 핵심 기능을 한 번씩 손으로 써봅니다. 둘, 프로그램이 커질 때 ‘레이어로 쪼개 지침으로 넘겨받는’ 설계를 익혀요 — 2차에서 배운 I/O 계약을 코드로 구현하는 겁니다. 셋, 이게 진짜인데요, 다음에 무슨 프로그램을 만들든 그대로 시작할 ‘파일 구조 한 장’을 가져갑니다.

강사 팁

셋째(스캐폴드)에 힘주기. 오늘 강의의 진짜 차별점 = ‘재사용 구조’ 예고.

→ 넘어가며: “그럼 왜 **Claude Code**인지부터.”

브라우저를 닫으면, 아무것도 안 남았다

3분

★ KEY POINT

Artifacts는 빠르지만 모래상자다 — 내 파일을 못 만지고, 닫으면 사라진다.

이렇게 말하세요

Artifacts 정말 편했죠. 근데 한계가 있어요. 내 컴퓨터 파일을 못 만지고, 브라우저 닫으면 끝이에요. “내 다운로드 폴더 정리해줘” 같은 진짜 일은 못 합니다. 그걸 하는 게 Claude Code예요. 내 컴퓨터에서 진짜로 돌고, 내 폴더를 다루고, 결과가 남습니다.

참고

가이드 01 ‘Claude Code란?’과 연결. 여유 있으면 “터미널에서 도는 AI 코딩 도구”라고 한 줄.

→ 넘어가며: “정확히 뭐가 다른지 표로 보죠.”

Artifacts vs Claude Code — 모래상자 vs 내 컴퓨터

4분

★ KEY POINT

Claude Code의 맥락엔 ‘내 컴퓨터’가 들어간다 — 그래서 내 진짜 파일을 다룰 수 있다.

이렇게 말하세요

한 표로 정리할게요. 도는 곳, 파일 개수, 내 파일 접근, 잘하는 일. 핵심은 가운데 줄 — 내 폴더·파일을 직접 만질 수 있다는 거예요. 이 한 줄이 오늘 강의 전체를 가능하게 합니다.

구분	Artifacts(4차)	Claude Code(6차)
어디서 도나	브라우저 모래상자	내 컴퓨터에서 진짜로
내 파일 접근	못 함	내 폴더·파일 직접
잘하는 일	빠른 웹 시제품	진짜 도구·자동화·공유

→ 넘어가며: “그 도구, 어떻게 첫 단추를 끼우나. 명령어부터.”

막히면 /, 외울 건 네 개

4분

★ KEY POINT

/help · /init · /clear · /permissions — 핵심 네 개. 여기에 대화가 길어지면 **/compact**(요약해 토큰 절약).

이렇게 말하세요

Claude Code엔 슬래시 명령어가 있어요. 다 외울 필요 없고 네 개만. 막히면 /help, 새 폴더 시작하면 /init(지침 자동 생성), 새 작업이면 /clear(대화 비우기). 위험한 변경 전엔 plan(계획) 모드로 — Shift+Tab 으로 켜거나, 쉽게는 그냥 ‘먼저 계획만 보여줘’라고 하면 돼요. 그리고 대화가 길어져 느려지거나 토큰이 아까우면 /compact — 지금까지를 요약해 가볍게 만들어줘요. 두 가지 꿀팁도 — !명령은 셀 바로 실행, @파일은 그 파일 꼭 집어 첨부.

강사 팁

실제로 /help 한 번 쳐서 화면을 보여주면 체감이 큼니다. 외우라고 강요 말고 “막히면 /help”만 각인.

→ 넘어가며: “**이제 진짜로 만들어봅시다. 만능 5단계로.**”

어떤 작업이든, 이 5단계

4분

★ KEY POINT

폴더 준비 → **claude** → 의도 설명 → 단계별 → 확인·반복. 이 흐름이면 무엇이든 시작한다.

이렇게 말하세요

모든 작업에 통하는 5단계입니다. 폴더 만들고, 그 안에서 claude 실행하고, 내가 뭘 원하는지 설명하고, 한 번에 다 시키지 말고 단계별로, 결과 보고 다시 부탁. 이 중 제일 중요한 게 세 번째, 의도 설명이에요. 무엇을·어떻게·왜를 한 문장에 담으면 절반은 끝납니다.

참고

가이드 05 ‘초보자 워크플로우’와 동일. 칠판에 5단계 적어두고 강의 내내 가리키면 좋음.

→ 넘어가며: “그 ‘의도 설명’을 우리가 아는 공식으로 — **R-PCCO.**”

코드 말고, R-PCCO로 묘사한다

5분

★ KEY POINT

코딩 맥락에선 ‘맥락(C)’에 어떤 환경·무슨 파일·어떻게 실행이 들어간다.

이렇게 말하세요

코드 한 줄도 안 씁니다. 우리가 아는 R-PCCO로 묘사만 해요. 역할은 ‘간단한 Node 도구 만드는 개발자’, 목적은 ‘폴더 파일 이름을 규칙대로 제자리에서 바꾸기’, 맥락은 ‘Windows·Node·로컬 서버’, 제약은 ‘미리보기·한 글 안전·단순하게’, 출력은 ‘node server.js로 열리는 한 페이지’. 이걸 그대로 던지면 됩니다.

강사 팁

구조설계안 8-1 프롬프트를 화면에 띄우고 실제로 입력. 파일이 생성되는 걸 함께 지켜보세요.

→ 넘어가며: “만드는 동안 화면에 뜨는 세 단어를 봅시다.”

Context · Tools · Permissions

4분

★ KEY POINT

이 세 단어를 이해하면 **Claude Code**의 모든 동작이 한 번에 보인다.

이렇게 말하세요

세 단어만 기억하세요. Context는 지금 Claude가 아는 모든 것(대화·파일·지침). Tools는 컴퓨터에 손대는 능력(Read·Write·Bash). Permissions는 안전벨트 — 위험한 작업 전에 묻습니다. 방금 파일 만들 때 ‘이거 해도 돼요?’ 뜯 거, 그게 Permissions예요. 빨리 가려고 다 Yes 누르지 마세요. 1초만 읽고.

참고

가이드 07 ‘핵심 개념’. 권한 다 Yes는 신규 실수 1위(가이드 12).

→ 넘어가며: “만들었으면 바로 돌려봐야죠.”

만들었으면 바로 돌려본다

4분

★ KEY POINT

Claude가 Bash로 서버를 켜고, 나는 브라우저로 확인한다 — Artifacts엔 없던 순간.

이렇게 말하세요

“로컬 서버로 띄워서 확인하자”고 하면, Claude가 알아서 Node.js로 server.js를 돌립니다. 그리고 브라우저에서 localhost:3000을 열면(여기서 3000은 ‘포트’, 프로그램의 문 번호예요) — 내 컴퓨터에서 진짜 도는 거예요. 이 ‘어, 진짜 돈다!’ 순간이 오늘의 첫 번째 감동 포인트입니다.

강사 팁

포트 충돌 나면 “다른 포트로 띄워줘”. 브라우저를 직접 띄워 화면을 같이 보세요.

→ 넘어가며: “돌아가는 동안 화면에 뜬 ‘도구’ 표시를 읽어봅시다.”

지금 애가 뭘 하는지 보인다

4분

★ KEY POINT

화면의 **Read·Write·Bash** 표시를 읽으면 **Claude**의 행동이 다 보인다.

이렇게 말하세요

화면에 Read, Write, Bash 같은 게 뜨죠? 그게 Claude가 지금 뭘 하는지예요. 파일 읽고(Read), 만들고(Write), 서버 돌리고(Bash). 그리고 꿀팁 — !ls 치면 폴더 바로 보고, @파일명 하면 그 파일 설명해줘요.

참고

가이드 14 ‘알면 편한 기능’. !·@는 한 번 시연하면 다들 따라 합니다.

→ 넘어가며: “**이제 이 프로그램을 안전하게 키워봅시다.**”

위험한 작업은 미리보기·계획부터

5분

★ KEY POINT

안전이 기능보다 먼저 — 되돌릴 수 없는 일은 미리보기·**plan**·되돌리기로 감싼다.

이렇게 말하세요

우리가 만드는 건 진짜 파일 이름을 바꾸는 프로그램이에요. 잘못 돌리면 내 파일이 엉망이 됩니다. 그래서 항상 ‘미리보기 → 확인 → 적용 → 되돌리기’. 큰 변경은 plan 모드로 — 아무것도 안 바꾸고 계획만 먼저 보여달라고 하세요. 이건 기능이자 습관입니다.

강사 팁

일부러 “미리보기 없이 막 돌리면 위험하다”를 체감시키면 다음 개선의 동기가 됩니다.

→ 넘어가며: “그 개선들을 한 번에? 아니요, 대화로 하나씩.”

한 방이 아니라, 대화로 키운다

4분

★ KEY POINT

Claude Code의 진짜 힘은 기존 코드를 고쳐 나가는 반복 개선이다.

이렇게 말하세요

기본 버전에 하나씩 없습니다. “미리보기 표 보여줘” → “되돌리기 넣어줘” → “규칙을 메뉴로 골라” 이렇게요. 매번 돌려보면서요. 한 번에 다 시키면 컨텍스트가 꽉 차서 품질이 떨어져요. 작게 요청 → 돌려보고 → 다음. 이게 핵심 리듬입니다.

참고

가이드 12 ‘큰 작업 한 번에 시키지 말기’. 작업 바뀌면 /clear.

→ 넘어가며: “그러다 보면 꼭 만나는 친구가 있죠. 에러.”

에러는 적이 아니라 정보다

5분

★ KEY POINT

잘 전달하면 5분 안에 해결. 디버깅 5요소 = 명령·에러 전체·환경·언제부터·기대 결과.

이렇게 말하세요

에러 뜨면 당황하지 마세요. 에러는 정보예요. “안 돼요 고쳐주세요”는 정보가 0이라 Claude가 추측해야 해요. 대신 다섯 가지를 주세요. 무슨 명령을 했는지, 에러 메시지 전체, 환경(OS·버전), 언제부터, 그리고 원하는 결과. 이러면 5분이면 고쳐줍니다.

강사 팁

일부러 없는 폴더 경로를 넣어 에러를 띄우고, 5요소로 고치는 걸 시연하면 가장 잘 남습니다.

→ 넘어가며: “자, 여기서 오늘의 핵심. 프로그램이 커졌어요.”

한 파일에 다 엉켰다

2분

★ KEY POINT

규칙·미리보기·적용·되돌리기가 한 곳에 — 복잡해졌다. 2차에서 배운 그 원리를 꺼낼 때.

이렇게 말하세요

자 보세요, 한 파일에 규칙도 있고 미리보기도 있고 적용도 있고 되돌리기도 있어요. 점점 엉켰죠? 수정하려면 어디를 건드려야 할지 헷갈리고요. 2차 강의에서 이거 해결하는 법 배웠습니다. 기억하세요?

강사 팁

이 슬라이드는 ‘문제 체감’용. 짧게 긴장 주고 다음 장에서 원칙으로 풀어줍니다.

→ 넘어가며: “2차의 그 한 문장 — 체인의 성패는 접점에 달렸다.”

체인의 성패는 ‘점점’에 달렸다

5분

★ KEY POINT

한 단계 = 한 책임. 단계 사이는 **JSON**으로만. 산문은 체인의 적.

이렇게 말하세요

2차에서 그랬죠 — “체인의 성패는 지침이 아니라 지침 사이의 점점에 달렸다.” 프로그램도 똑같아요. 일이 커지면 단계로 나누고, 한 단계는 한 가지 책임만 집니다. 수집은 변환 안 하고, 변환은 저장 안 해요. 그리고 단계 사이엔 반드시 구조화된 데이터, **JSON**으로 넘깁니다. 두루뭉술한 산문으로 넘기면 다음 단계가 못 받아요.

참고

2차 I-MRKO ‘지침 체인 설계’. ‘한 AI = 한 책임’ 원칙의 코드 버전.

→ 넘어가며: “그럼 우리 파일 변경기를 세 단계로 쪼개봅시다.”

수집 → 변환계획 → 출력

5분

★ KEY POINT

세 단계로 쪼개고 사이엔 **JSON** 계약. 한 덩어리 코드가 역할별 파일로 — 레이어가 눈에 보인다.

이렇게 말하세요

수집은 폴더를 읽어 파일 목록(JSON)을 만들고, 변환계획은 규칙대로 새 이름 계획(JSON)을 짜고, 출력은 그 계획대로 진짜 이름을 바꿉니다. 사이에 흐르는 게 ‘계약’이에요. 이렇게 나누면 한 덩어리였던 게 collect·transform·output 파일로 갈라져요. Artifacts는 이걸 단일 파일에 묶여넣어야 했죠. 멀티파일, 이게 Claude Code의 가치입니다.

강사 팁

화면에 src 폴더가 모듈로 갈라지는 걸 보여주면 ‘레이어’가 추상이 아니라 실물로 와닿습니다.

→ 넘어가며: “그 ‘계약’에 정확히 뭘 적느냐 — 포·필·범·메.”

접점은 포·필·범·메로 고정

4분

★ KEY POINT

포(포맷)·필(필수항목)·범(범위)·메(메타). 두 지침에 같은 포맷을 적어야 접점이 안 깨진다.

이렇게 말하세요

계약에 적는 네 가지예요. 포 — 어떤 구조로 넘길지(JSON). 필 — 반드시 들어갈 항목. 범 — 개수나 분량 한계. 메 — 충돌 표시 같은 메타정보. 핵심은, 넘기는 쪽 지침과 받는 쪽 지침에 ‘같은 포맷’을 적어둬야 한다는 거예요. 안 그러면 2차에서 배운 그 실수 — 포맷 안 맞아서 다음 단계가 깨집니다.

참고

2차 ‘포·필·범·메’ 그대로. 구조설계안 §3에 실제 JSON 예시 있음.

→ 넘어가며: “그래서 우리는 지침을 ‘먼저’ 씁니다.”

지침을 먼저 쓰고, 그 계약대로 만든다

5분

★ KEY POINT

스펙 우선(spec-first) — 각 단계의 입출력 계약을 MD로 먼저 못 박고 구현한다.

이렇게 말하세요

순서가 거꾸로예요. 코드 먼저가 아니라 지침 먼저. specs 폴더에 01_수집·02_변환·03_출력 지침을 써요. 각 지침엔 역할, 입력/출력 계약(포·필·범·메), 책임, 절대 안 하는 것. 그리고 Claude에게 “이 지침대로 각 단계를 별도 파일로 구현해줘” 한마디. 그럼 묘사 한 번에 멀티파일이 만들어집니다.

강사 팁

구조설계안 8-2 프롬프트를 그대로 입력. “지침 MD 먼저 보여주고 구현해줘”의 순서를 꼭 강조.

→ 넘어가며: “이 지침들, 더 단단하게 만드는 두 가지.”

지침은 스킬로, 중간엔 검증을

4분

★ KEY POINT

자주 쓰는 절차는 스킬로 재사용, 단계 중간엔 검증을 끼워 오류 전파를 막는다.

이렇게 말하세요

두 가지만 더. 하나, 자주 하는 절차(예: 커밋 규칙)는 스킬로 저장하면 다음 프로그램에서 또 써요. 둘, 변환 다음에 ‘빈 이름·중복’ 검증 단계를 끼웁니다. 2차에서 배웠죠 — 3단계 이상이면 검증 하나는 꼭. 앞 단계 오류가 뒤로 번지지 않게요.

참고

가이드 17 Skills. 검증 단계 = 2차 ‘체인 중간 검증’ 원칙.

→ 넘어가며: “자, 여기서 오늘 가장 중요한 자산. 재사용 구조.”

어떤 프로그램이든 이 뼈대

3분

★ KEY POINT

프로그램 = 코드 + 구조 파일들. 다음에 무엇을 만들든 이 폴더 뼈대로 시작한다.

이렇게 말하세요

오늘 진짜 가져가실 건 파일 변경기가 아니라 이 구조예요. CLAUDE.md, README, DEVLOG, HANDOVER, .env, specs, .claude, src. 어떤 프로그램을 만들든 이 뼈대로 시작하면 혼자서도 안 무너져요. 초보가 무너지는 건 실력이 아니라 구조가 없어서거든요.

참고

‘03 표준구조 가이드’가 이 강의의 핵심 산출물. 복붙 템플릿 전부 거기에.

→ 넘어가며: “하나씩 무슨 역할인지 봅시다.”

있으면 안 무너지는 파일들

5분

★ KEY POINT

CLAUDE.md·specs는 AI가 자동으로 읽고, **DEVLOG·HANDOVER**는 내 미래를 위한 기록이다.

이렇게 말하세요

CLAUDE.md는 폴더에 두기만 하면 Claude가 매번 자동으로 읽어요 — 신입사원 매뉴얼이죠. specs는 단계 계약. README는 사람용 설명서. DEVLOG는 ‘무엇을 언제 왜 했나’ 개발일지 — 다음에 열어도 1초에 복원돼요. HANDOVER는 ‘현재 상태와 다음 할 일’ — 세션 끊겨도 이어가요. 그리고 .env는 비밀, .gitignore로 깃에서 제외.

강사 팁

‘자동으로 읽힘(CLAUDE.md)’ vs ‘내가 읽는 기록(DEVLOG·HANDOVER)’ 구분을 명확히. 2차 I-MRKO ‘AI에게 한 장, 사람에게 한 장’ 복습.

→ 넘어가며: “언제 뭘 만들고 어떤 명령어를 쓰는지 정리해드릴게요.”

언제 어떤 명령어를 쓰나

4분

★ KEY POINT

새 폴더 **/init**, 새 작업 **/clear**, 위험 전 **plan**. 그리고 처음부터 다 만들지 말 것.

이렇게 말하세요

명령어는 타이밍이에요. 새 폴더 시작하면 **/init**, 새 작업이면 **/clear**, 위험한 변경 전엔 **plan**(계획) 모드 (Shift+Tab, 또는 ‘먼저 계획만 보여줘’). 그리고 파일도 처음부터 다 만들지 마세요. 최소 3개 (CLAUDE·README·.gitignore)로 시작해서, 필요할 때 DEVLOG·specs 추가하고, 더 커지면 HANDOVER·스킬. 작게 심고 키웁니다.

참고

최소 → 권장 → 풀. 가이드 08+ ‘프로젝트 파일 설계’.

→ 넘어가며: “**이제 망쳐도 안전하게, git.**”

AI가 망쳐도 한 줄로 되돌린다

4분

★ KEY POINT

git = 시간 여행. 큰 작업 전엔 항상 커밋해 안전망을 친다.

이렇게 말하세요

“AI가 내 코드 망치면 어쩌지?” 그래서 git이 있어요. 작업 전에 git commit 한 번 해두면, AI가 100줄 고쳐놔어도 마음에 안 들면 git restore 한 줄로 원상복구. “이거 git으로 저장하고 GitHub에 올려줘” 하면 커밋부터 원격까지 Claude가 도와줍니다.

참고

가이드 11 GitHub. ‘작업 전 commit’이 신규 실수 5위(백업 안 함)의 해법.

→ 넘어가며: “그럼 이걸 남들도 쓰게 하려면? 여기 함정이 있어요.”

이건 URL로 남이 못 쓴다 — 왜?

5분

★ KEY POINT

무엇을 만드느냐가 배포 방식을 정한다. 내 파일 만지는 도구는 **GitHub**로 코드 공유.

이렇게 말하세요

이걸 Vercel에 올리면 남이 쓸 수 있을까요? 안 돼요. 배포된 웹서버는 방문자 컴퓨터의 파일을 못 만지거든요. 그래서 이런 도구는 GitHub에 코드를 올려, 남이 받아서 자기 컴퓨터에서 실행하는 식으로 공유해요. 반대로 4차 운세앱처럼 남의 파일 안 만지는 건 Vercel URL 배포가 맞고요. 무엇을 만드느냐가 배포 방식을 정합니다.

강사 팁

이건 ‘솔직한 배포 수업’. 한계를 숨기지 말고 왜 그런지 이해시키면 오히려 인상 깊습니다.

→ 넘어가며: “자, 이제 직접 해볼 차례.”

이제 직접 따라 만들어봅시다

12분

★ KEY POINT

‘따라하기 명령어’를 그대로 입력하며 묘사 → 테스트 → 레이어 → 스캐폴드 → 공유 한 사이클을 돈다.

이렇게 말하세요

이제 여러분 차례예요. 나눠드린 ‘따라하기 명령어’를 그대로 Claude Code에 입력하면 됩니다. 빈 폴더에서 묘사로 시작해서, 로컬로 돌려보고, 레이어로 쪼개고, 스캐폴드 파일 갖추고, git까지. 막히면 두 문장만 기억하세요 — “큰 변경이니 계획만 먼저”, “이 에러 5요소로 줄게, 고쳐줘”.

강사 팁

조교가 막힌 수강생 개별 지원. 완성본(04 샘플)은 정답지로만, 먼저 보여주지 말 것.

→ 넘어가며: “다 해보셨으면, 오늘 흐름을 한 장으로 정리.”

프로그램 만들기, 이 순서로

3분

★ KEY POINT

묘사 → 테스트 → 디버깅 → 레이어 → 스캐폴드 → **git** → 공유. 한 사이클이 곧 실력.

이렇게 말하세요

오늘 한 걸 순서로 묶으면 이래요. 폴더 만들고 init, 묘사로 작게 시작, 로컬 테스트, 2단계 넘으면 레이어와 지침, 예러는 5요소, 스캐폴드 파일 갖추고, 커밋하고 공유. 이 한 사이클을 외우면 어떤 프로그램이든 같은 흐름으로 만들 수 있어요.

→ 넘어가며: “그리고 이것 다음에도 쓰라고 — 체크리스트 한 장.”

다음 프로그램도 이 한 장으로 시작한다

4분

★ KEY POINT

이 체크리스트가 이 강의가 남기는 한 장. 다음 프로그램도 이걸로 시작한다.

이렇게 말하세요

이 체크리스트를 캡처해 두세요. 새 프로그램 만들 때마다 그대로 따라가시면 됩니다. init, 묘사, 로컬 테스트, 2단계 이상이면 specs, .env, DEVLOG·HANDOVER, 반복작업은 스킴, 커밋, 공유 결정. 오늘 만든 건 도구 하나지만, 진짜 배운 건 이 ‘시작하는 법’이에요.

강사 팁

03 가이드 §8 체크리스트를 띄워두고 “이 한 장이 오늘의 결론”이라고 못 박기.

→ 넘어가며: “습관 세 가지로 압축할게요.”

오늘부터, 이 세 가지

3분

★ KEY POINT

① R-PCCO로 묘사·2단계 넘으면 레이어 ② 미리보기·plan·git ③ 작게→돌려보고→다음.

이렇게 말하세요

세 가지만 가져가세요. 하나, 묘사는 R-PCCO로, 2단계 넘으면 레이어로 쪼개고 단계별 지침. 둘, 위험하거나 되돌릴 수 없는 일은 미리보기·plan·git 먼저. 셋, 한 방에 다 시키지 말고 작게 요청해서 돌려보고 다음으로. 작업 바뀌면 /clear.

→ 넘어가며: “오늘 우리가 손으로 만진 기능, 한눈에.”

오늘 손으로 만진 Claude Code

3분

★ KEY POINT

하루 만에 명령어·도구·권한·로컬서버·**plan**·디버깅·레이어·**MD**·스킬·**git**·공유를 다 써봤다.

이렇게 말하세요

정리하면, 오늘 여러분은 슬래시 명령어부터 도구 관찰, 권한, 로컬 서버, 계획 모드, 디버깅, 레이어 설계, 지침 MD, 스킬, git, 공유까지 — Claude Code의 큰 줄기를 다 손으로 밟았어요. 이게 하루치라는 게 놀랍지 않나요?

→ 넘어가며: “여기서 더 가면 뭐가 있느냐.”

각 레이어를 서브에이전트로

3분

★ KEY POINT

오늘의 ‘한 책임’ 레이어가 진짜 ‘AI 팀’이 되는 다음 단계 — **MCP·Hooks·서브에이전트**.

이렇게 말하세요

오늘 단계를 모듈로 나눴죠. 다음엔 각 단계를 ‘서브에이전트’로 만들어 진짜 AI 팀처럼 굴릴 수 있어요. 그리고 MCP로 외부 도구 연결, Hooks로 안전장치. 가이드 고급편에 다 있습니다. 오늘 배운 방식 그대로, 더 강해집니다.

참고

가이드 15 MCP · 16 Hooks · 18 서브에이전트.

→ 넘어가며: “오늘은 여기까지. 질문 받겠습니다.”

고맙습니다 · Q&A

5분

★ KEY POINT

묘사 → 테스트 → 레이어 → 스캐폴드 → 공유. 이제 혼자서도 시작할 수 있다.

이렇게 말하세요

오늘 묘사만으로 내 컴퓨터에서 도는 도구를 만들고, 레이어로 쪼개고, 구조를 갖추고, 공유까지 해봤습니다. 가장 중요한 건 '재사용 구조 한 장' — 다음 프로그램은 이걸로 혼자 시작하시면 됩니다. 질문 받을게요. 고맙습니다.

강사 팁

Q&A는 '무엇을 만들고 싶은지' 거꾸로 물어 다음 강의 수요를 수집하기 좋은 타이밍.

→ 끝.